Metodi per il ritrovamento delle informazioni

Alessandro Carella

Indice

1. Introduzione 4
   1.1. Gestire la conoscenza 4
   1.2. Acquisizione della conoscenza 4
      1.2.1. Data Retrieval 5
      1.2.2. Information Retrieval 6
      1.2.3. Data mining 6
      1.2.4. Il processo KDD 6
      1.2.5. Text Mining 6
   1.3. Ritrovamento delle informazioni intelligente 8
      1.3.1. Ritrovamento delle informazioni 8
      1.3.2. Come le persone cercano informazioni 8
   1.4. Sistemi di ritrovamento delle informazioni 9
      1.4.1. Rilevanza 9
      1.4.2. Ritrovamento delle informazioni intelligente 10
      1.4.3. Natural Language Processing 11
      1.4.4. Machine learning 11
2. Boolean and Vector Space Retrieval Models 12
   2.1. Retrieval Models 12
   2.2. Definizione formale dei modelli IR 12
   2.3. Modello di ritrovamento booleano 13
      2.3.1. Vettori di incidenza 13
      2.3.2. Indice invertito 14
      2.3.3. Fasi dell'Indicizzatore 15
      2.3.4. Elaborazione della query: Operazione AND 16
      2.3.5. Query booleane: corrispondenza esatta 17
      2.3.6. Problemi del modello booleano 17
   2.4. Fasi di preprocessing 18
      2.4.1. Tokenizzazione 18
      2.4.2. Lemmatizzazione 22
      2.4.3. Stemming 23
   2.5. Punteggio, Pesatura dei Termini e Modello dello Spazio Vettoriale 23
      2.5.1. Problemmi del modello di ritrovamento booleano 23
      2.5.2. Ranked Retrieval Models 24
      2.5.3. Classificazione come base del ranked retrieval 24
      2.5.4. Bag of words model 26
      2.5.5. Term frequency tf 26

2.5.6. Document frequency 27
2.5.7. Documenti come vettori 28
2.5.8. Query come vettori 29

Introduzione

1.1. Gestire la conoscenza

Gestire la conoscenza significa raccogliere la conoscenza, organizzarla, distruibuirla e renderla accessibile a chi ne ha bisogno (nel momento e nel posto in cui serve) al fine di:

- risparmiare tempo,
- migliorare la qualità dei servizi,
- ridurre i tempi di accesso all'informazione ed alla fruizione dei servizi.

La conoscenza è un capitale:

- intangibile,
- volatile,
- difficile da concretizzare e conservare.

Circa il 90% dei dati presenti nei database del mondo è in forma non strutturata. Gli obiettivi sono quelli di costruire sistemi in grado di processare documenti in linguaggio naturale e ritrovare conoscenza da basi di dati in forma testuale.

1.2. Acquisizione della conoscenza

Il processo di acquisizione della conoscenza dal testo in linguaggio naturale è chiamato **text mining**. La classificazione delle tecniche di estrazione ed esplorazione dei dati si basa sull'incrocio di due dimensioni fondamemtali: l'obiettivo dell'utente e la natura dei dati analizzati.

- L'approccio dell'utente:
  - **Search (Goal-oriented)**: l'utente ha un obiettivo e sa cosa sta cercando. Formula una query per recuperare informazioni o documenti specifici.
  - **Discover (Opportunistic)**: l'utente ha un approccio esplorativo. Non cerca un dato esatto noto a priori, ma vuole «scoprire» pattern nascoti, tendenza, anomalie o estrarre nuova conoscenza dei dati.
- Il formato dei dati:
  - **Structured Data**: Dati strutturati, tipicamente organizzati in schemi rigidi.
  - **Unstructured Data**: Dati non strutturati, composti prevalentemente da testo libero in linguaggio naturale, privi di uno schema predefinito.

Incrociando queste dimensioni si ottengono quattro diverse aree di studio e applicazione:

- **Data Retrieval (Dati Strutturati + Search)**: consiste nel recuperare record esatti da un database strutturato. Si basa su logica deterministica: un record corrisponde o non corrisponde alla query,
- **Information Retrieval (Dati non Strutturati + Search)**. È la ricerca di informazioni all'interno di collezioni di documenti testuali. Poichè il testo non è strutturato, il sistema restituisce un ranking di documenti in base al loro grado di «rilevanza» rispetto alla query.
- **Data Mining (Dati strutturati + Discover)**. È l'applicazione di algoritmi statistici e di macchine di learning su grandi database strutturati per scoprire pattern ricorrenti, regole di associazione o per addestrare modelli predittivi.
- **Text Mining (Dati non strutturati + Discover)**. È il processo di derivazione di informazioni di alta qualità, pattern e nuova conoscenza partendo da enormi quantità di testo libero.

Tutto ciò è mostrato nella seguente figura.

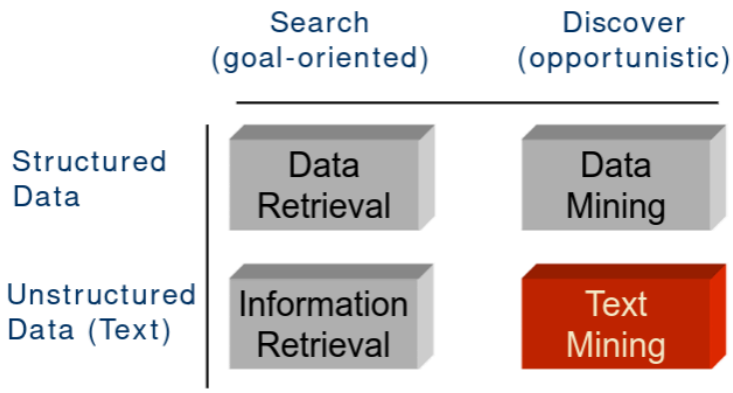


Figura 1: Processo di acquisizione della conoscenza

1.2.1. Data Retrieval

Come detto in precedenza, il processo di data retrieval è il ritrovamento di un record in un database strutturato.

Database Type	Structured
Search Mode	Goal-driven
Atomic entity	Data Record
Example Information Need	"Find a Japanese restaurant in Boston that serves vegetarian food."
Example Query	"SELECT * FROM restaurants WHERE city = boston AND type = japanese AND has_veg = true"

Figura 2: Data Retrieval

1.2.2. Information Retrieval

Il processo di information retrieval è la ricerca di un'informazione rilvante in una sorgente di dati non strutturati (tipicamente in formato testo).

Database Type	Unstructured
Search Mode	Goal-driven
Atomic entity	Document
Example Information Need	"Find a Japanese restaurant in Boston that serves vegetarian food."
Example Query	"Japanese restaurant Boston" or Boston->Restaurants->Japanese

Figura 3: Information Retrieval

1.2.3. Data mining

Il processo di data mining permette di scoprire nuova conoscenza attraverso l'analisi di dati.

Database Type	Structured
Search Mode	Opportunistic
Atomic entity	Numbers
Example Information Need	"Show trend over time in # of visits to Japanese restaurants in Boston "

Figura 4: Data Mining

1.2.4. Il processo KDD

Il processo KDD, acronimo per *Knowledge Discovery from Databases*, è un processo non banale di identificazione valida, nuova e potenzialmente, e ultimente di comprensibili pattern nei dati.

1.2.5. Text Mining

Il processo di text mining è la scoperta di nuova conoscenza attraverso l'analisi del testo.

Database Type	Unstructured
Search Mode	Opportunistic
Atomic entity	Language feature or concept
Example Information Need	"Find the types of food poisoning most often associated with Japanese restaurants"
Example Query	Rank **diseases** found associated with "Japanese restaurants"

Figura 5: Text Mining

Il processo di text mining si può dividere nei seguenti passi:

1. **Text Preprocessing (Pre-elaborazione del testo)**. Coinvolge l'analisi *sintattica* e *semantica* per preparare i dati in un formato leggibile dalle fasi successive.
2. **Features Generation (Generazione delle Feature / Trasformazione del testo)**. Consiste nel trasformare il testo pre-elaborato in una rappresentazione matematica o strutturata. L'approccio principale è il **Bag of words** che ignora l'ordine delle parole e la grammatica, concentrandosi slla presenza e la molteplicità dei termini.
3. **Features Selection (Selezione delle feature)**. La fase precedente genera uno spazio vettoriale di dimensioni enormi, questa fase selezione solo le caratteristiche più rilevanti e informative. Si basa su tecniche come:
   - **simple counting**: metodi basati sul conteggio semplice delle frequenze dei termini.
   - **statistics:** valutazioni statistiche per capire quali feature hanno il maggiore potere discriminante.
4. **Text/data mining**. È il cuore del processo, dove si applicano gli algoritmi di macchine di machine learning ai dati ormai strutturati e filtrati. Si divide in due paradigmi:
   - **Classification (Supervised learning)**. Apprendimento supervisionato, dove si addestra un modello su testi già etichettati per classificare nuovi documenti.
   - **Clustering (Unsupervised learning)**. Apprendimento non supervisionato, dove l'algoritmo raggruppa autonomamente i documenti in base alle loro similarità intrinseche, senza avere categorie predefinite.
5. **Analyzing result (Interpretazione e valutazione)**. L'ultima fase in cui i pattern estratti vengono analizzati e interpretati per estrarre il vero e proprio «valore» o «conoscenza». Questa fase prevede un **meccanismo di feedback**. L'interpretazione dei risultati può portare a rivalutare e modifcare i parametri di qualsiasi delle fasi precedenti per migliorare l'output finale.

Il text mining è importante anche nell'impresa, in quanto il processo di estrazione di conoscenza, precedentemente sconosciuta, da fonti testualii è utilizzabile per prendere decisioni aziendali. Inoltre, permette di organizzare e di categorizzare scoprendo tendenza e apprendendo concetti. Inoltre, il text minining nell'impresa è anche necessario in quanto scoprire tendenze, idee, opinioni e gusti degli utenti sta diventando sempre più impegnativo. Ci sono tante fonti da analizzare come e-mail, forum, articol, ecc, e l'obiettivo è quello di analizzare migliaia di testi in pochi secondi raggruppandoli in funzione del loro contenuto.

1.3. Ritrovamento delle informazioni intelligente

1.3.1. Ritrovamento delle informazioni

Il ritrovamento delle informazioni, oppure *information Retrieval, IR*, si occupa della rappresentazione, dell'archiviazione, organizzazione e accesso a elementi informativi, come documenti, pagine Web, cataloghi online, record strutturati e oggetti multimediali. L'ambito di ricerca iniziale nell'area del ritrovamento delle informazioni riguardava l'indicizzazione del testo e la ricerca di documenti utili in una collezione. Cercare pagine nel web è un compito non banale in quanto bisogna prima ritrovare documenti relativi ad una data query e poi ritrovare, in modo efficiente, informazioni da un grande set di documenti.
Il compito del ritrovamento delle informazioni è quello di trovare una classifica di un insieme di documenti che sono relativi ad una data query. Data la query dell'utenet l'obiettivo di un sistema di ritrovamento delle informazioni è quello di ritrovare informazioni utili o rilevanti per l'utente. Il sistema IR deve classificare le informazioni in accordo con un grado di rilevanza alla query data dall'utente.

1.3.2. Come le persone cercano informazioni

L'interazione degli utenti con le interfacce di ricerca è influenzata da diversi fattori. Nel campo dell'information retrieval, si fa, inoltre, una distinzione fondamentale tra le modalità con cui si cerca l'informazione.
Il modo in cui un utente interagisce con un'interfaccia di ricerca varia in funzione di tre elementi:

- **il tipo di compito**: l'obiettivo pratico che l'utente deve portare a termine.
- **l'esperienza nel dominio**: il livello di conoscenza pregressa che chi cerca l'informazione possiede riguardo allo specifico argomento trattato.
- **tempo e sforzo**: la quantità di tempo e di impegno cognitivo che l'utente ha a disposizione o è disposto a inverstire nel processo.

Come detto prima, si opera una distinzione principale tra due approcci alla ricerca:

- **information lookup**, ricerca mirata;
- **exploratory search**, ricerca esplorativa.

I compiti basati sull'information lookup presentano le seguenti caratteristiche:

- sono assimilabili al recupero di dato di fatto o alla ricerca di risposte dirette a domande specifiche;
- il bisogno dell'utente può essere soddisfatto ottenendo pezzi discreti di informazioni, come numeri, date, siti Web, ecc.;
- questa modalità si sposa molto bene con i paradigmi di interazione standard offerti dai classici motori di ricerca Web.

L'exploratory search, invece, è divisa in compiti di apprendimento e compiti investigativi. La ricerca di apprendimento richiede più di singole coppie query-risposta e richiede a chi effettua le ricerca di spendere tempo per: analizzare e leggere molteplici elementi di informazione e per sintetizzare i contenuto in una nuova forma comprensibile.
Il compito investigativo riguarda un processo a lungo termine che:

- coinvolge interazioni che prendono luogo in un periodo di tempo prolungato;
- può restituire risultati che necessitano di essere valutati criticamente prima di essere integrati nel personale bagaglio professionale e di conoscenza;
- può occuparsi di trovare una grande proporzione di informazioni rilevanti disponibili.

La ricerca di informazioni può essere vista come parte di un processo più grande al quale ci si riferisce come *sensemaking*. Il processo di **sensemaking** è un processo iterativo che consiste nel formulare una rappresentazione concettuale a partire da una vasta collezione di dati. All'interno di questo processo, la maggior parte dello sforzo cognitivo e del tempo dell'utente viene investita nella sintesi, ovvero nella creazione di una «buona rappresentazione» strutturata dell'argomento.
Le attività di sensemaking possono svilupparsi secondo diverse modalità operative:

- **interwoven search, ricerca intrecciata**. In alcuni casi, l'attività di ricerca si intreccia continuamente con l'analisi lungo tutto l'arco del processo.
- **batch process, ricerca a blocchi**. In altri casi, il processo è diviso in fasi distinte: l'utente esegue prima un blocco intensivo di sole ricerche, che viene poi seguito da un blocco separato dedicato interamente all'analisi e alla sintesi dei risultati raccolti.

Degli esempi di compiti di analisi profonda che richiedono l'utilizzo del processo di sensemaking sono:

- **Legal discovery**: il processo in ambito legale di ricerca, raccolta e analisi di grandi quantità di documenti o prove elettroniche.
- **Epidemiologia**: il tracciamento e lo studio della diffusione delle malattie.
- **Analisi del customer care**: lo studio sistematico dei reclami dei clienti con l'obiettivo di migliorare il servizio offerto.
- **Business intelligence**: l'acquisizione e l'analisi di dati complessi per ricavare informazioni strategiche e supportare le decisioni aziendali.

1.4. Sistemi di ritrovamento delle informazioni

1.4.1. Rilevanza

La rilevanza è un giudizio soggettivo e può includere:

- essere sul giusto argomento;

- avere informazioni recenti;
- soddisfare gli obiettivi dell'utente e del suo uso previsto delle informazioni.

La definizione più semplice di rilevanza è che la query appare parola per parola nel documento. Una nozione più generale, invece, è che le parole nella stringa appaiono frequentemente nel documento, in qualsiasi ordine. Per questo si usa il termine di *bag of words*, intesa come una sacca di parole del quale non importa l'ordine.
Il problema di queste parole chiave è che è possibile non ritrovare documenti rilevanti che includono, però, dei sinonimi di tali parole. Oppure, è possibile ritrovare documenti irrilevanti che possono contenere termini polisemici, ovvero stessa parola ma con significato diverso. Ad esempio, la parola «bat» in inglese può significare sia pipistrello che la mazza da baseball.

1.4.2. Ritrovamento delle informazioni intelligente

Un sistema di *intelligent IR* si distingue dai sistemi di base per le seguenti caratteristiche:

- prende in considerazione il *significato* delle parole utilizzate;
- prende in considerazione l'*ordine* delle parole all'interno della query;
- si adatta all'utente basandosi su un feedback che può essere diretto o indiretto;
- nel processo di *relevance feedback*, il sistema raccoglie il feedback fornito, genera una nuova query e ripete l'operazione di redcupero delle informazioni.

Analizziamo l'architettura di un sistema IR:

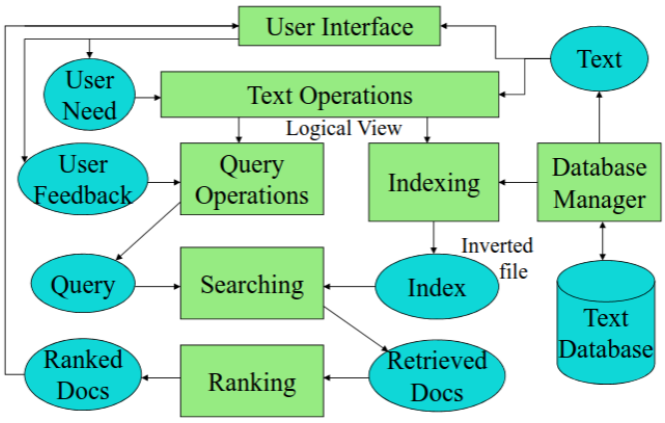


Figura 6: Architettura di un sistema IR

Dalla foto precedente descriviamo le seguenti componenti:

- **Text operations**, le operazioni sul testo. Questa fase si occupa di formare le parole indice, ovvero i *token*. Include operazioni come:
  - la rimozione delle stopword
  - lo stemming, ovvero ridurre le parole alle loro radici tramite la rimozione di prefissi e suffissi.

- **Indexing**, l'indicizzazione. Questa fase consiste nel costruire un *indice invertito* detto *inverted index* che mappa le parole verso i puntatori dei documenti in cui esse compaiono.
- **Searching**, la ricerca. Questa fase ha lo scopo di recuperare, a partire dall'inverted index, i documenti che contengono uno specifico token di ricerca.
- **Ranking**. Questa fase assegna un punteggio a tutti i documenti recuperati basandosi su una metrice di rilevanza. Questa componente può occuparsi anche del raggruppamento, ovvero trovare elementi in comune per presentare all'utente dei gruppi di documenti.

Le aree di ricerca dei sistemi di intelligent IR riguardano il processing del linguaggio naturale e il machine learning.

1.4.3. Natural Language Processing

Il Natural Language Processing, NLP, nell'ambito dell'Information Retrieval si caratterizza per i seguenti aspetti:

- è focalizzato sull'analisi sintattica, semantica e pragmatica del testo e del discorso in linguaggio naturale.
- la capacità di analizzare la sintassi e la semantica potrebbe consentire un recupero delle informazioni basato sul significato, piuttosto che sulle semplici parole chiave.

L'applicazione del Natural Language Processing al campo dell'Information Retrieval si sta sviluppando lungo diverse direzioni di ricerca principali:

- **Word Sense Disambiguation**. Sviluppo di emtodi per determinare il significato corretto di una parola ambigua basandosi sul contesto in cui si trova.
- **Information Extraction**. Utilizzo di metodi volti a indentificare specifiche porzioni di informazione all'interno di un documento.
- **Question answering**. Creazione di metodi per fornire risposte a specifiche domande poste in linguaggio naturale ricavandole da ampi corpi di documenti.

1.4.4. Machine learning

Il campo del Machine Learning si concentra sui seguenti aspetti:

- È focalizzato sullo sviluppo di sistemi computazionali che sono in grado di migliorare le proprie prestazioni attraverso l'esperienza.
- **Apprendimento supervisionato (Supervised learning)**. Prevede la classificaione automatica di esempi basandosi sull'apprendimento di concetti derivati da esempi di addestramento precedentemente etichettati.
- **Apprendimento non supervisionato (Unsupervised learning)**. Consiste nell'utilizzo di metodi automatici per raggruppare esempi non etichettati all'interno di gruppo che risultino significativi.

Boolean and Vector Space Retrieval Models

2.1. Retrieval Models

Un modello di ritrovamento, detto **retrieval model**, specifica i dettagli di:

- rappresentazione dei documenti;
- rappresentazione delle query
- funzione di ritrovamento, detta **retrieval function**.

Per implementare la funzione di ritrovamento bisogna determinare la nozione di rilevanza che può essere binaria o continua.
Inoltre, determina anche la funzione di classificazione, detta **ranking function** che assegna i punteggi ai documenti relativi ad una data query.

2.2. Definizione formale dei modelli IR

Definiamo formalmente i modelli di ritrovamento, *IR models*, come segue:

> **Definizione 1** *(Modello IR)* **.** Un modello di ritrovamento delle informazioni è una quadrupla $[D, Q, F, R(q_i, d_j)]$ ove:
> 1. D è un insieme di viste logiche per i documenti nella collezione.
> 2. Q è un insieme di viste logiche per le informazioni che servono all'utente. Queste viste sono chiamate *query*.
> 3. F è un framework per la modellazione dei documenti e query.
> 4. $R(q_i, d_j)$ è una funzione di ranking, che confronta la query con il documento e rileva quanto il documento è rilevante rispetto alla query data.

Esistono diversi modelli di ritrovamento, come mostrari nella figura di seguito:

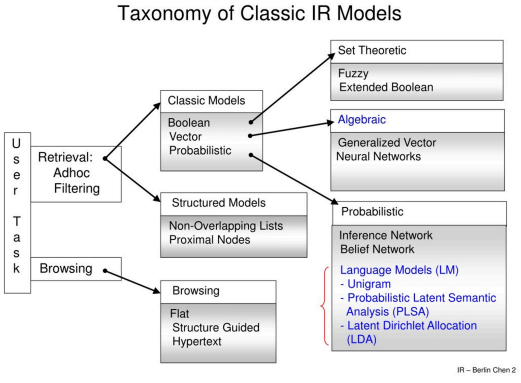


Figura 7: Tassonomia dei modelli di ritrovamento

Cercare foto simile a quella nelle slide

2.3. Modello di ritrovamento booleano

Il modello di ritrovamento booleano è basato sulla teoria degli insiemi. Per approcciarsi a questi modelli ci sono dei passi comuni nella fase di preprocessing:

- rimuovere caratteri non voluti, come tag HTML, punteggiature, numeri, ecc.
- dividere il documento in *token* dividendolo sulla base degli spazi;
- ricondurre i token alla radice, detta operazione di *stemming*
- rimuovere comuni parole chiuse, come congiunzioni, articoli, ecc.
- Individuare frasi comuni, possibilmente usando un dizionario specifico di dominio
- costruire gli indici invertiti, detti *inverted index*, che permettono di accedere alla lista dei documenti che contengono una data parola, detta *keyword*.

Nel modello di ritrovamento booleano un documento è rappresentato mediante un **insieme** di parole e le query sono espressioni booleane di keyword collegate tramite le operazioni di AND, OR e NOT, includendo l'uso di parentesi graffe per indicare lo scopo. In output si ottiene se il documento è rilevante o meno, non ci sono corrispondenze parziali o una classificazione di rilevanza. Quindi la nozione di rilevanza in questo modello è di tipo booleana.

2.3.1. Vettori di incidenza

Consideriamo la seguente tabella:

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1

worser	1	0	1	1	1	0

Quindi, per ogni termine abbiamo un vettore composto da 0 o 1. Per rispondere ad una query come:

Brutus, Ceaser and NOT Calpurnia

si prendono i vettori per le rispettive parole:

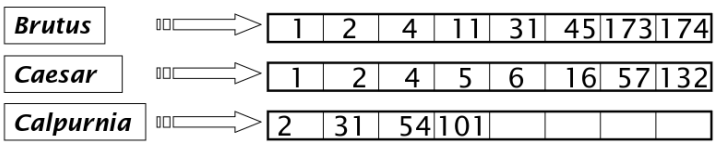
- *Brutus*: 110100;
- *Caesar*: *110111*;
- *Calpurnia* (complementato): 101111.

Infine, eseguiamo l'and bit a bit e otteniamo:

$$110100 \quad \text{AND} \quad 110111 \quad \text{AND} \quad 101111 = 100100$$

2.3.2. Indice invertito

Per ogni termine t, memorizziamo una lista di tutti i documenti che contengono tale ptermine. Ogni documento è identificato da un **docID** è un numero seriale per i documenti.
Supponiamo di implementare gli indici invertiti utilizzando array di dimensione statica e consideriamo la seguente figura:



Se, ad esempio, si aggiunge la parola *Caesar* al documento 14, non possiamo inserire quest'ultimo nell'array e, quindi, bisogna usare strutture dati in grado di supportare liste di **postings** a dimensione variabile.
In memoria centrale, questo problema si risolve tipicamente utilizzando:

- **Liste concatenate**
- **Array a lunghezza variabile**.

Queste soluzioni dinamiche permettono di aggiungere nuovi **docID** man mano che l'indice cresce, accettando dei compromessi tra l'ottimizzazione dello spazio occupato e la facilità di inserimento. Su disco, invece, è normale e ottimale mantenere una sequenza continua di dati per velocizzare i tempo di lettura.
Inoltre, a prescindere dalla struttura dinamica scelta in memoria, è fondamentale ricordare che i documenti all'interno di ogni lista di **posting** devono sempre essere mantenuti ordinati per **docID** crescente per garantire l'efficienza delle operazioni di intersezione durante le query.
Segue un esempio di costruzione dell'indice invertito:

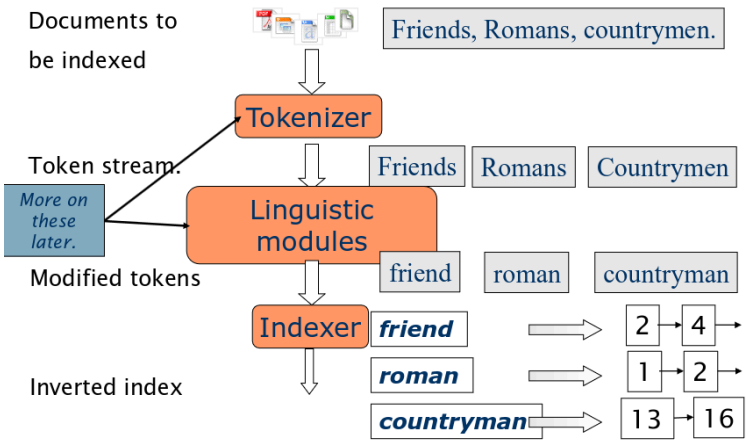


Figura 9: Costruzione dell'indice invertito

2.3.3. Fasi dell'Indicizzatore

Analizziamo le passi che compongono l'indicizzatore:

1. Creazione delle coppie (Token, docID)
   - Il processo estrae il testo e genera una sequenza formata da coppie di valori: un **token modificato** e l'**ID del documento** in cui quel token compare.
   - I token vengono definiti «modificati» perchè, subiscono una pre-elaborazione prima di essere inseriti nella tabella.
2. **Ordinamento**. Una volta generata la sequenza iniziale di coppie (Token, docID), il sistema procede con l'ordinamento di questi dati. Questo passaggio è identificato come la fase centrale e più importante del processo di indicizzazione.
   L'algoritmo riorganizza l'intera tabella applicando due criteri di ordinamento in sequenza:
   1. **Ordinamento primario per termine**: tutte le coppie vengono ordinate alfabeticametne in base alla colonna del token. In questo modo, tutte le occorrenze della stessa parola vengono raggruppate vicine.
   2. **Ordinamento secondario per docID**: a parità di termine, le tuple vengono ordinate in modo crescente in base all'identificativo del documento.
3. **Dizionario e postings**. Dopo aver ordinato alfabeticametne i termini e i relativi docID, il sistema trasforma la lista lineare nella struttura finale dell'indice invertito attraverso tre operazioni fondamentali:
   - **Unione dei termini (Merging)**: le entrate multiple dello stesso termine relative a un singolo documento vengono unite. Se una parola compare più volte, non si creano puntatori duplicati allo stesso documento, ma le occorrenze vengono compattate.

- **Divisione in due strutture (Split)**: I dati elaborati vengono formalmente suddivisi nelle due componenti principali: il **Dizionario** e le relative liste di **Postings**.
- **Aggiunta della Document Frequency**: Per ogni termine nel dizionario viene aggiunta l'informazione sulla *Document Frequency*. Questo valore indica il numero totale di documenti distinti all'interno della collezione in cui quel termine è presente.

2.3.4. Elaborazione della query: Operazione AND

Per comprendere come il sistema estrae i documenti a partire da un indice invertito, consideriamo l'elaborazione di una query con operatore logico AND: **Brutus AND Caesar**.

Le fasi dell'elaborazione sono le seguenti.

1. Localizza il termine **Brutus** all'interno del Dizionario e ne recupera la rispettiva lista di *postings*.
2. Localizza il termine **Caesar** nel Dizionario e ne recupera la relativa lista di *postings*.
3. Esegue un Merge delle due liste di postings per individuare i documenti che contengono contemporaneamente entrambe le parole.

Affinchè l'intersezione sia computazionalmente efficiente, sussistono due vincoli fondamntali:

- I postings all'interno di ciascuna lista devono essere rigorosamente ordinati per docID;
- Le liste devono essere processate in **ordine di lunghezza crescente**, iniziando dalle parole più rare per ridurre da subito il numero di documenti da confrontare.

L'algoritmo di intersezione è il seguente.

ALGORITMO 1 — *Intersezione*

```
1:  Intersect(p1, p2)
2:      answer ← ⟨⟩
3:      while p1 ≠ NIL and p2 ≠ NIL
4:          do if docID(p1) = docID(p2)
5:              then ADD(answer, docID(p1))
6:              p1 ← next(p1)
7:              p2 ← next(p2)
8:          else if docID(p1) < docID(p2)
9:              then p1 ← next(p1)
10:             else p2 ← next(p2)
```

```
11:  | return answer
```

2.3.5. Query booleane: corrispondenza esatta

Il modello di ritrovamento booleano si basa sulla capacità di sottoporre al sistema delle query formulate come vere e proprie espressioni booleane.

1. **Caratteristiche del modello**.
   - Le query booleane utilizzano gli opearatori logici fondamentali.
   - In questo modello, ogni documento viene visto e trattato matematicamente come un semplice *insieme di parole*.
   - La sua natura è **precisa**: l'esito della ricerca è puramente binario. Un documento soddisfa esattamente la condizione dettata dalla query oppure non la soddisfa affatto.
   - Per via di queste caratteristiche, rappresenta forse il modello più semplice su cui sia possibile costruire un sistema di Information Retrieval.
2. **Rilevanza storica e attuale**
   - È stato lo strumento di retrieval principale in ambito commerciale per circa tre decenni.
   - Ancora oggi, molti dei sistemi di ricerca di uso comune si basano su logico booleana. Tra questi figurano la ricerca all'interno delle emeail, i cataloghi delle biblioteche e la funzione Spotlight di Mac OS X.

2.3.6. Problemi del modello booleano

Nonostante la loro semplicità, i modelli di retrieval booleano presentano diverse criticità e limitazioni:

- **Estrema rigidità**: l'utilizzo dell'operatore AND richiede che tutti i termini siano presenti, portando spesso a ottenere troppi pochi risultati. Al contrario, l'operatore OR si accontenta di un qualsiasi termine, rischiando di generare una mole eccessiva di risultati.
- **Difficoltà nell'esprimere richieste complesse**. Risulta complicato esprimere i bisogni informativi complessi degli utenti, poichè queste informazioni devono essere obbligatoriamente tradotte e forzate all'interno di un'espressione booleana.
- **Mancanza di controllo sul volume dei risultati**. È difficile controllare il numero di documenti che il sistema recupera, poichè tutti i documenti che presentano una corrispondenza verranno inevitabilmente restituiti.
- **Difficoltà nel ranking**. Non è possibile generare una classifica di rilevanza accurata, dato che tutti i documenti recuperati soddisfano logicamente la query allo stesso livello.
- **Difficoltà nell'applicazione del relevance feedback**. Risulta problematico implementare meccanismi di feedback. Se l'utente identifica un documento resti-

tuito come rilevante o irrilevante, non è chiaro come la query originale debba essere modificata di conseguenza.

2.4. Fasi di preprocessing

Quando un sistema si appresta a elaborare un documento greszzo, deve prima determinare alcune sue caratteristiche fondamentali, rispondendo a tre domande:

- **Formato**: in quale formato si trova il file;
- **Lingua**: in quale lingua è scritto il testo;
- **Codifica**: quale codifica dei caratteri viene utilizzata.

Dal punto di vista formale, ognuna di queste sfide rappresenta un vero e proprio **problema di classificazione**. Nonostante la loro natura, nella pratica questi compiti vengono spesso affrontati in modo *euristico*: invece di addestrare modelli complessi, la classificazione viene predetta applicando delle regole semplici. Un esempio di questa implementazione è il riconoscimento della lingua: se nel testo ci sono molte occorrenze della parole «the», allora il documento è in inglese.
Durante il processo di parsing e indicizzazione, i sistemi devono affrontare diverse complicazioni legate alla varietà di formati e lingue:

1. **la gestione del multilinguismo**
   - la collezione di documenti da indicizzare può includere testi scritti in molte lingue differenti. Questo implica che un singolo indice potrebbe trovarsi a memorizzare e gestire termini appartenenti a diverse lingue contemporaneamente
2. **documenti e componenti ibridi**. In alcuni casi, un singolo documento o i vari componenti che lo formano possono contenere al loro interno molteplici lingue o formati. Un tipico esempio è un'email scritta in france che ha come allegato un documento PDF scritto in tedesco.
3. **Il problema dell'unità documentale**. Alla luce di queste complessità strutturali, sorge una domanda fondamentale: come si definisce esattametne un «documento unitario» da indicizzare? Ci sono diverse interpretazioni a seconda del contesto:
   - è un singolo file?
   - è un'email?
   - è un'email considerata insieme a tutti i suoi allegati?
   - oppuer un gruppo di file interconnessi?

2.4.1. Tokenizzazione

La tokenizzazione è il processo di suddivisione del testo grezzo in unità fondamentali e significative. Analizziamone un esempio.

Esempio 1. Sia data in input la frase:

Friends, Romans and Countrymen

e in output il sistema elabora il testo e restituisce un insieme di tokens. In questo esempio specifico vengono estratti:

Friends Romans Countrymen

Diamo una definizione formale di token.

Definizione 2 *(Token)* **.** Un **token** è definito come un'istanza di una specifica sequenza di caratteri all'interno del testo.

Ogni token così generato diventa un candidato potenziale per diventare una voce vera e propria all'interno dell'indice. Tuttavia, per diventare una voce definitiva dell'indice, il token deve prima subire un **ulteriore processamento**.
La domanda findamentale che guida le fasi successive della progettazione di un indicizzatore è: **quali sono i token validi da emettere?**
Nel processo di tokenizzazione sorgono diverse problematiche e ambiguità su come interpretare correttamente i caratteri speciali, i trattini e la punteggiatura. I casi critici principali sono i seguenti:

- **Gestione degli apostrofi e dei genitivi**. Prendendo come esempio la frase «Finland's capital», come deve essere gestito il termine possessivo? Il token risultante deve essere *Finland?, Finlands?* oppure *Finland's?*
- **Parole con trattino**. Espressioni come «Hewlett-Packard» devono essere suddivise in due token distinti? Ci sono casi come «state-of-the-art», in cui bisogna decidere se spezzare o meno una sequenza complessa unita da trattini. Esistono parole ambigue come «co-education», o variazioni ortografiche come *lowercase*, *lower-case* e *lower case*. In questi casi, può rilevarsi efficace fare in modo che sia l'utente stesso a inserire eventuali trattini per guidare la ricerca.
- **Nomi composti**. Nomi propri come «San Francisco» devono essere considerati come un unico token oppure come token separati? E soprattutto: con quale criterio logico si deicde se un'espressione plurinominale costituisce un token unico?

La gestione dei numeri nella tokenizzazione
La gestione delle sequenze numeriche e dei codici alfanumerici rappresenta un aspetto critico durante la tokenizzazione, a causa della grande varietà di formati esistenti.
Ad esempio le date possono essere rappresentate come *3/20/91*, *Mar. 12, 1991* oppure *20/3/91*.
Spesso queste stringhe contengono spazi interni che complicano la separazione dei token. I sistemi di IR meno recenti potrebbero scegliere di non indicizzare affatto

i numeri. Tuttavia, indicizzare i numeri è spesso estremamente utile: basti pensare alla ricerca sul web di codici di errore o di *stack trace*. Una possibile soluzione tecnica a questo problema è l'uso degli **n-grammi**. Molto spesso, i metadati associati vengono indicizzati separatamente come **meta-data**.

Problematiche legate alla lingua

Le scelte di tokenizzazione variano profondamente in base alle specifità linguistiche e morlogiche della lingua in cui è scritto il documento.

1. **Caso di lingua francese**:
   - **Gestione delle elisioni**: un'espressione come l'*ensemble* solleva il dubbio se debba essere considerata come un token unico oppure come due distinti. Le opzioni di suddivisione includono: separare in *L* e *ensemble*?, considerare *L'*? oppure espandere in *Le* e *ensamble*.
   - **Problemi di corrispondenza (matching)**. Spesso si desidera che la ricerca di *l'ensemble* restituisca una corrispondenza anche con forme simili come un *esemble*. TUttavia, su Google questo comportamento non era garantito fino al 2003, evidenziando le grandi sfide legate all'**internazionalizzazione**.
2. **Caso del Tedesco**:
   - **Composti nominali non segmentati**. In lingua tedesca i sostantivi composti vengono scritti unizi senza spazi. Un esempio stano è la parola *Lebensversicherungsgeselischaftangestellter*, che significa letteralmente «impiegato di una compagnia di assicurazioni sulal vita».
   - **L'uso del Compound Splitter**. Per ovviare a questo rpoblema, i sitemi di Information Retrieval in lingua tedesca traggono enormi vantaggi dall'integrazione di un modulo di scompozione noto come **compound splitter**. L'adozione di qeusto modulo può incrementare le prestazioni di ricerca per il tedesco fino al 15%.
3. **Caso di cinese e giapponese**:
   - Assenza di spazi: il cinese e il giapponese non prevedono sapzi tra le parole;
   - Tokenizzazione non unica: a causa di ciò, non è sempre garantita una tokenizzazione univoca;
   - Nel caso del giapponese ci sono *alfabeti multipli mescolati*: la gestione diventa più complessa a causa dellapresenza contemporanea di più alfabeti intrecciati tra loro. Le date e gli importi si presentano in formati differenti. Inoltre, l'utente finale può scegliere di esprimere la propria query intermante utilizzando l'alfabeto hiragana.
4. **Caso di arabo e ebraico**
   - *Direzione di scrittura*: l'arabo e l'ebraico si scrivono fondamentalmente da destra a sinistra, sebbene determinati elementi come i numeri siano scritti da sinistra a destra.

- *Rappresentazione e Unicode*: grazie all'utilizzo di Unicode, la presentazione visiva di superficie risulta complessa a causa della mescolanza di direzioni, ma la forma memorizzata sottostante rimane lineare e diretta.

Stop words

Con una stop list, puoi escludere completamente dal dizionario le parole più comuni, in quanto hanno un contenuto semantico piccolo e ce ne sono tante.
La tendenza attuale però tende ad includere nel dizionario anche le stop word. Questo perchè sono state sviluppate tecniche di compressione efficaci e lo spazio richiesto per includere le stop words nel sistema è diventato estremamente ridotto. Inoltre ci sono avanzate tecniche di ottimizzazione che fanno si che il costo computazionale al momento della query sia minimo.
Infine, ci sono dei casi in cui le stop words sono necessarie:

- *Ricerche per frase*: risultano indispensabili per query specifiche come «King of Denmark»
- Titoli di opere o canzoni: sono essenziali per gestire stringhe come «Let it be».
- *Query «relazionali»*: servono per interpretare correttamente ricerche come «flights to London».

Normalizzazione dei termini

La normalizzazione dei termini è necessaria sia per le parole presenti nel testo indicizzato e sia per le parole inserite nelle query in una forma comune. L'obiettivo è, ad esempio, fare in modo che il sistema riconosca e metta in corrispondenza forme equivalenti come *U.S.A* e *USA*.
Il risultato del processo di normalizzazione prende il nome di **termine**, inteso come una parola normalizzata (*word type*) che rappresenta una singola voce all'interno del dizionario del sistema di Information Retrieval.
Il processo definisce implicitamente delle classi di equivalenza tra i termini. Questo avviene operativamente attraverso regole quali:

- La rimozione dei punti per formare un termine standard ad esempio *U.S.A* e *USA* appartengono alla classe di equivalenza $[USA]$.
- La rimozione dei trattini, ad esempio *anti-discriminatory* e *antidiscriminatory* appartengono alla classe di equivalenza [antidiscriminatory].

Inoltre, ci sono lingue in cui anche gli accenti possono diventare critici. Ad esempio, il termine francese *résumé* rispetto a *resume*. Queste forme differenti dovrebbero essere trattate come equivalenti dal sistema.
Il fattore più importante da considerare nella progettazione è il modo in cui gli utenti tendono a digitare le proprie query per queste parole. Anche nelle lingue che prevono standardmente l'uso di accenti, gli utenti spesso non li digitano quando effetuano una ricerca. Di conseguenza, è spesso preferibile normalizzare i termini riconducibili a una forma priva di eccenti.

Anche le date e l'alternanza nell'uso giapponese dei caratteri *kana* rispetto ai caratteri cinesi devono essere normalizzati. Il processo di tokenizzazione e normalizzazione dipende strettamente dalla lingua e risulta pertanto profondamente intrecciato con il rilevamento della lingua stessa. Ad esempio nella frase «Morgen will ich in MIT» bisogna stabilire se si tratta effettivamente della preposizione tedesca *mit*.
La regola fondamentale è quella di normalizzare sia il testo che viene inserito nell'indice sia i termini specifici della query, riconducendoli esattamente alla medesima forma standard.
Un'altra operazione fondamentale è quella della riduzione delle maiuscole, e ci sono diversi modi per farlo:

- *Conversione universale*: prevede di ridurre tutte le lettere a minuscole;
- *Eccezioni contestuali*: si valutano le maiuscole a metà frase, come nel caso di *General Motors*, o la distinzione tra acronimi e parole comuni.

Risulta spesso ottimale applicare la conversione in minuscolo a tutto, poichè gli utenti tendono a digitare in minuscolo a prescindere dalla capitalizzazione formalmente corretta.
Ci si interroga su quale sia l'effetto concreto delle operazioni di normalizzazione sulle prestazioni del sistema in termini di precisione e richiamo.
Un'alternativa alle classi di equivalenza è quella di includere nel dizionario tante varianti di un termine e poi fare un'espansione asimettrica a query time. Ad esempio se l'utente inserisce la query *window* il sistema cerca per *window, windows*. Questo approccio è potenzialmente più potente ma meno efficiente.

Thesauri e Soundex

Analizziamo la gestione di sinonimi e omonimi. È possibile gestirli tramite classi di equivalenza costruite manualmente, come ad esempio l'equivalenza tra *Car* e *automobile*, oppure tra *color* e *colour*. Si possono riscrivere i termini per formare classi di equivalenza: quando un documento contiene la parola *automobile*, ad esempio, viene indicizzato sotto la forma combinata *car-automobile* (e viceversa). Un'alternativa consiste nell'espandere la query in fase di ricerca, facendo in modo che se la query contiene *automobile*, il sistema cerchi anche sotto *car*.
Per gestire gli errori di ortografia, invece, si applica l'approccio *soundex*, un sistema che raggruppa le parole in classi di equivalenza basandosi su euristiche di tipo fonetico.

2.4.2. Lemmatizzazione

Il processo di lemmatizzazione consiste nel ridurre le forme flesse o varianti alla loro forma base, ovvero la forma che si cercherebbe all'interno di un dizionario. Esempi di lemmatizzazione sono:

- le voci verbali *am, are, is* vengono ricondotte a *be*;

- i termini *car, cars, car's, cars'* vengono ricondotti a *car*.

Un esempio di trasformazione di una frase è: «the boy's cars are different colors» che diventa «the boy car be different color».
Diamo una definizione formale di lemmatizzazione:

> **Definizione 3** *(Lemmatizzazione)* **.** La lemmatizzazione implica l'esecuzione di una riduzione «corretta» alla forma del lemma presente come intestazione nel dizionario.

2.4.3. Stemming

Il processo di stemming consiste nel ridurre i termini alle «radici» prima di procedere con l'indicizzazione. Lo stemming suggerisce un'operazione grossolana di rimozione dei suffissi, la cui implementazione è dipendente dalla lingua. Ad esempio, parole derivate come *automate, automatic* e *automation* vengono tutte ridotte alla forma comune *automat*.
Analizziamone un impatto pratico sul testo:

for example compressed and compression are both accepted as equivalent compress

venga trasformata in forme alterate quali

for exampl compress and compress ar both accept as equival to compress

Algoritmo di Porter

L'algoritmo di Porter è l'algoritmo più diffuso per lo stemming della lingua inglese. I risultati suggeriscono che sia almeno tanto valido quanto le altre opzioni di stemming disponibili. Il processo si basa su convezioni e su 5 fasi di riduzione.

- Le fasi vengono applicate in modo sequenziale.
- Ciascuna fase è costituita da un insieme di comandi.
- Una convenzione tipica stabilisce che, tra le regole presenti in un comando composto, si debba selezionare quella che si applica al suffisso più lungo.

Analizziamone un esempio pratico: la parola *Girls*, la lettera «s» viene identificata come suffisso, mentre *Girl* costituisce la radice.

2.5. Punteggio, Pesatura dei Termini e Modello dello Spazio Vettoriale

2.5.1. Problemmi del modello di ritrovamento booleano

Fino ad ora, le nostre query sono state booleane, quindi i documenti potevano essere rilevanti o meno. Il modello di ritrovamento booleano è utile per utenti esperti che hanno una conoscenza completa dei loro bisgoni e della collezione di documenti. È anche utile per le applicazioni che possono facilmente consumare migliaia di

risultati.
Non risulta efficiente, però, per la maggior parte degli utenti in quanto non capaci di scrivere query booleano (oppure se lo sono, pensano che sia troppo impegnativo). Inoltre, la maggior parte degli utenti non vogliono guardare tra migliaia di risultati. Le query booleane spesso risultano in troppi pochi risultati oppure in troppo risultati. Ci vogliono grandi capacità per costruire una query che produce un numero di documenti rilevanti ragionevoli, in quanto le operazioni di AND ne producono pochi e l'operazione di OR ne producono tanti.

2.5.2. Ranked Retrieval Models

I modelli di ritrovamento basati su ranking, anzichè restituire un insieme non ordinato di documenti che soddisfano un'espressione booleana, il sitema restituisce un ordinamento sui migliori documenti della collezione rispetto a una specifica query.
Inoltre, invece di utilizzare un linguaggio di interrogazione basato su operatori ed espressioni formali, la query dell'utente è costituita semplicemente da una o più parole scritte in linguaggio naturale. In linea di principio si possono distinguere due scelte separate, ovvero il linguaggio di interrogazione (*query language*) e il modello di ritrovamento (*retrieval model*). Tuttavia, nella pratica, i modello di ritrovamento basati sul ranking sono normalmente associati alla query a testo libero.
Quando un sistema produce una classifica di risultati, un insieme di risultati troppo grande non è più un problema, in quanto ne mostriamo i primi $k(\approx 10)$.

2.5.3. Classificazione come base del ranked retrieval

L'obiettivo dei sistemi di ranking e restituire in ordine i documenti che hanno la maggiore probabilità di risultare utili per l'utente che effettua la ricerca. Sorge spontanea la domanda su come sia possibile ordinare per grado di importanza i documenti presenti nella collezione in relazione a una determinata query.
Si assegna un punteggio, ad esempio compreso nell'intervallo [0, 1], a ciascun documento. Questo punteggio misura quanto bene un documento e una query riescono a «corrispondere».
Ci serve un modo per assegnare un punteggio ad un documento. Per iniziare la trattazione, si prende in considerazione una query composta da un unico termine. Se il termine della query non compare all'interno del documento il punteggio deve essere pari a 0. A tal proposito ci si chiede il motivo di questa regola e se sia possibile adottare soluzioni migliori. Più il termine della query è frequente all'interno del documento, maggiore dovrebbe essere il punteggio assegnato.

Primo approccio: Coefficiente di Jaccard

Il coefficiente di Jaccard rappresenta una misura comunemente impiegata per calcolare la sovrapposizione tra due insiemi, A e B.

Il coefficiente si calcola come il rapporto tra la cardinalità dell'intersezione e la cardinalità dell'unione dei due insiemi, espresso dalla formula:

$$jaccard(A, B) = \mid A \cap B \mid \setminus \mid A \cup B \mid$$

Analizziamone alcune proprietà:
- $jaccard(A, B) = 1$ nel caso di insiemi identici;
- $jaccard(A, B) = 0$ nel caso in cui l'intersezione sia nulla.

Gli insiemi non devono necessariamente avere la stessa dimensione e il coefficiente assegna sempre un valore compreso tra 0 e 1.

Esempio 2. Viene posto l'interrogativo su quale sia il punteggio di corrispondenza tra query e documento calcolato dal coefficiente di Jaccard per due documenti specifici. Sia data la seguente query:

$$Q : \text{ides of march}$$

I documenti sono:
- D_1 : caesar died in march
- D_2 : the long march.

Applichiamo la formula

$$jaccard(Q, D) = \mid Q \cap D \mid \setminus \mid Q \cup D \mid$$

e ottieniamo:
- $jaccard(Q, D_1) = \frac{1}{6}$
- $jaccard(Q, D_2) = \frac{1}{5}$

Il coefficiente di Jaccard non considera la **frequenza dei termini**. Inoltre, i termini rari all'interno di una collezione sono più informativi rispetto a quelli frequenti, ma il coefficiente di Jaccard ignora completamente questa informazione. Emerge la necessità di disporre di un metodo più sofisticato per effettuare la noramalizzazione basata sulla lunghezza del testo.

Secondo approccio: matrici di conteggio termini-documenti
Consideriamo il numero di volte in cui un determinato termine compare all'interno di un documento. Ciascun documento viene rappresentato come un vettore di conteggio appartenente a $\mathbb{N}^V$ corrispondente a una colonna della matrice.

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth

Antony	**157**	**73**	**0**	**0**	**0**	**0**
Brutus	**4**	**157**	**0**	**1**	**0**	**0**
Caesar	**232**	**227**	**0**	**2**	**1**	**1**
Calpurnia	**0**	**10**	**0**	**0**	**0**	**0**
Cleopatra	**57**	**0**	**0**	**0**	**0**	**0**
mercy	**2**	**0**	**3**	**5**	**5**	**1**
worser	**2**	**0**	**1**	**1**	**1**	**0**

2.5.4. Bag of words model

In questo modello si fa uso della rappresentazione vettoriale che non tiene conto dell'ordine delle parole all'interno di un documento. Ad esempio frasi come «John is quicker than Mary» e «Mary is quicker than John» generano esattamente gli stessi vettori.
Per certi versi, questo modello rappresenta un passo indietro rispetto all'indice posizionale, il quale era in grado di distinguere questi due documenti.

2.5.5. Term frequency tf

> **Definizione 4** *(Term frequency)* **.** La frequenza del termine $tf_{t,d}$ di un termine t in un documento d è definita come il numero di volte in cui t compare all'interno di d.

Vogliamo utilizzare il valore di tf nel calcolo dei punteggi di corrispondenza tra query e documento, ma ci si interroga su come farlo correttamente. La frequenza grezza in sè non è ciò che vogliamo. Un documento con 10 occorrenze di un termine è certamente più rilevante rispetto a un documento con una sola occorrenza, ma non è 10 volte più rilevante.
La rilevanza non cresce in modo direttamente proporozionale rispetto alla frequenza del termine. È bene notare che la frequenza, nell'ambito dell'Information Retrieval, è pari al conteggio puro.

Pesatura basata sulla frequenza logaritmica

Il peso basato sulla frequenza logaritmica di un termine t in un documento d è definito dalla funzione a tratti:

$$w_{t,d} = \begin{cases} 1 + \log_{10}(tf_{t,d}) & \text{se } tf_{t,d} > 0 \\ 0 & \text{altrimenti} \end{cases}$$

Ad esempio:
- $0 \to 0$
- $1 \to 1$
- $2 \to 1.3$
- $10 \to 2$
- $1000 \to 4$ e così via.

Per una coppia documento query il punteggio complessivo si ottiene sommando i pesi per tutti i termini t presenti sia nella query q che nel documento d:

$$score(d, q) = \sum_{t \in q \cap d} \big(1 + \log(tf_{t,d})\big)$$

La **condizione di annullamento**, ovvero se il punteggio è pari 0, si avvera se nessuno dei termini della query è presente all'interno del documento.
La formula precedentemente introdotta definisce il cosidetto ***sublinear tf-scaling***, in cui il peso è calcolato come

$$w_{t,d} = \begin{cases} 1 + \log_{10}(tf_{t,d}) & \text{se } tf_{t,d} > 0 \\ 0 & \text{altrimenti} \end{cases}$$

L'approccio normale, **normal** è il metodo più semplice e diretto e consiste nell'utilizzare il numero puro di occorrenze del termine all'interno del documento. Ma, come discusso in precedenza, l'utilizzo di una scala sublineare per la frequenza dei termini risulta complessivamente migliore e più efficace.

2.5.6. Document frequency

I termini rari, nell'intera collezione dei documenti, sono più informativi dei termini frequenti, ad esempio delle stop word.
Consideriamo un termine in una query che è raro nella collezione, ad esempio *aracnocentrico*. Un documento che contiene tale termine sarà, molto probabilmente, rilevante alla query «aracnocentrico». Quindi, dobbiamo assegnare un peso alto per termini considerati rari.
Generalmente i termini frequenti sono **meno informativi** dei termini rari.
VEDERE ESEMPI SLIDE

idf weight

df_t è la *frequenza del documento* del termine t: il numero di documenti che contiene tale termine t
- df_t è una misura inversa dell'informatività di t, più piccola è meglio è.
- $df_t \leq N$

Definiamo con *idf*, **inverse document frequency**, di t come

$$idf_t = \log\left(\frac{N}{df_t}\right)$$

è una funzione che dipende solo da t, non dal documento. Utilizziamo $\log\left(\frac{N}{df_t}\right)$ e non $\frac{N}{df_t}$ per attenuare l'effetto di *idf*.
Per ogni termine t nella collezione di documenti esiste il corrispettivo valore di *idf*. L'idf non ha effetto su query che presentano un'unico termine in quanto esiste un valore di idf per ogni termine nella collezione. Il valore di idf influisce sulla classificazione dei documenti per query che hanno almeno 2 termini.

Differenza fra collection e document frequency
La collection frequency di un termine t conta il numero di occorrenze di t nella collezione, contando occorrenze multiple. La document frequency, conta in quanti documenti è presente il termine t.
Per avere una ricerca migliore uniamo questi due concetti nel **tf-idf weighting**.

tf-idf weighting
Il **tf-idf weight** di un termine è il prodotto del suo *tf weight* e del suo *idf weight*:

$$w_{t,d} = \left(1 + \log\left(tf_{t,d}\right)\right) \times \log\left(\frac{N}{df_t}\right)$$

Questo è il migliore schema di pesatura nell'ambito dell'information retrieval. È bene notare che il segno «-» non indica sottrazione, ma un trattino. Ci sono altre nomenclature per questo schema, come **tf.idf** oppure **tf** × **idf**. Il tf-idf ha 2 proprietà:
1. Aumenta con il numero di occorrenze in un documento;
2. Aumenta con la rarità del termine nella collezione.

Con questo possiamo definire la classificazione finale dei documenti data una query con la seguente formula:

$$Score(d, q) = \sum_{t \in q \cap d} tf_{t,d} \times idf_t$$

2.5.7. Documenti come vettori
Supponiamo di avere uno spazio vettoriale con V dimensioni, ove:
- i termini sono gli assi dello spazio;
- i documenti sono i punti o i vettori nello spazio.

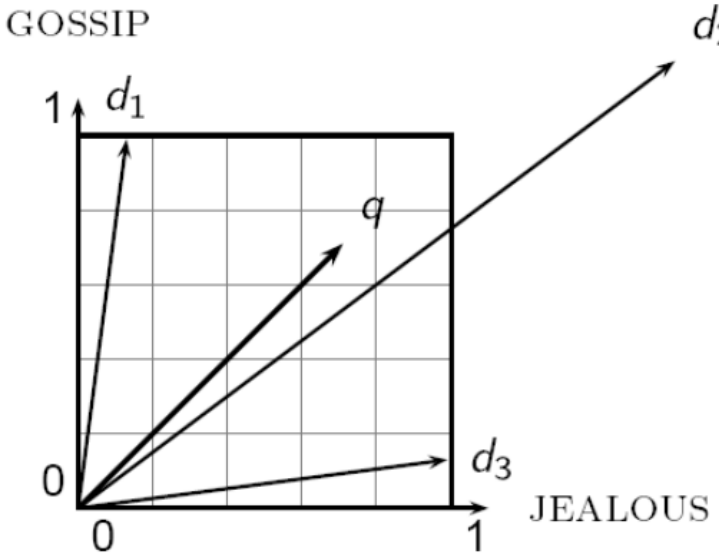
In un web search engine questo spazio vettoriale può arrivare anche ad avere una dimensione di decine di millioni.
Questi sono vettori molto scarsi in quanto la maggior parte delle voci è pari a 0.

2.5.8. Query come vettori

Vogliamo fare la stessa cosa fatta per i documenti, rappresentare le query come uno spazio vettoriale e vogliamo classificare i documenti secondo la loro vicinanza alla query nello spazio vettoriale. La vicinanza rappresenta la similarità dei vettori e l'inverso della distanza.
Stiamo volgendo la nostra attenzione per non far ricorso al modello di ritrovamento booleano e per classificare con un punteggio più alto i documenti più rilevanti rispetto ai documenti meno rilevanti.

Formalizzazione della prossimità nello spazio vettoriale

Un primo approccio è quello di considerare la distanza tra due punti, intesa come la distanza tra gli estremi dei due vettori.
Ci si interroga sulla possibilità di utilizzare la distanza euclidea ma è una cattiva idea perchè restituisce valori elevanti quando viene applicata a vettori con lunghezze differenti. Consideriamo il seguente esempio:



La distanza euclidea tra q e d_2 è grande anceh quando la distribuzione dei termini nella query q e la distribuzione dei termini nel documento d_2 sono molto simili.
La soluzione è quella di utilizzare l'angolo anzichè la distanza. Consideriamo il seguente esperimento: si prenda un documento d e lo si concaten a se stesso. Questo documento viene chiamato d'. Semanticamente d e d' possiedono esattamente lo stesso contenuto. La distanza euclidea tra i due documenti può risultare piuttosto grande. Se si utilizza la rappresentazione $tf_{t,d}$ l'angolo tra i due documenti è pari a 0, il che corrisponde alla massima somiglianza.
Quindi l'idea è quella di ordinare i documenti in base all'angolo che formano con la query.

Dagli angoli ai coseni

Le due seguenti modalità di ordinamento risultano equivalenti:

- ordinare i documenti in ordine *decrescente* rispetto all'angolo compreso tra la query e il documento;
- ordinare i documenti in ordine crescente rispetto al coseno tra query e documento ($\cos(\text{query}, \text{documenti})$)

La funzione coseno è monoticamente decrescente nell'intervallo $[0^\circ, 180^\circ]$.
Analizziamo come e perchè dovremmo calcolare i coseni.
Un vettore può essere normalizzato dividendo ciascun componente per la sua lunghezza, per questo utilizziamo la norma 2, L_2:

$$\|\vec{x}\|_2 = \sqrt{\sum_i x_i^2}$$

Dividendo un vettore per la sua norma 2 lo rende un vettore di lunghezza pari ad 1, quindi un **versore**. I due documenti d e d' dell'esperimento precedente avranno vettori identici dopo averli normalizzati. Quindi documenti lunghi e corti avranno un peso comparabile.
Consideriamo ora la formula $\cos(\text{query}, \text{document})$.

$$\cos\left(\vec{q}, \vec{d}\right) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}|\ |\vec{d}|} = \frac{\vec{q}}{|\vec{q}|} \cdot \frac{\vec{d}}{|\vec{d}|} = \frac{\sum_{i=1}^{|V|} q_i d_i}{\sqrt{\sum_{i=1}^{|V|} q_i^2} \sqrt{\sum_{i=1}^{|V|} d_i^2}}$$

Per vettori normalizzati, la similarità del coseno è semplicemente il prodotto scalare:

$$\cos\left(\vec{q}, \vec{d}\right) = \vec{q} \cdot \vec{d} = \sum_{i=1}^{|V|} q_i d_i$$

Esempio 3 *(Similarità del coseno tra 3 documenti)* **.** Consideriamo i tre documenti:

- *Sas: Sense and Sensibility*
- *Pap: Pride, and Prejudice*
- *WH: Wuthering Heights?*

e consideriamo la frequenza dei termini:

term	SaS	PaP	WH
affection	115	58	20
jealous	10	7	11
gossip	2	0	6
wuthering	0	0	38

Figura 11: Term frequency

Il peso calcolato tramite log frequency è il seguente:

term	SaS	PaP	WH
affection	3.06	2.76	2.30
jealous	2.00	1.85	2.04
gossip	1.30	0	1.78
wuthering	0	0	2.58

Figura 12: Log frequency

Normalizzando tali pesi otteniamo:

term	SaS	PaP	WH
affection	0.789	0.832	0.524
jealous	0.515	0.555	0.465
gossip	0.335	0	0.405
wuthering	0	0	0.588

Figura 13: Normalizzazione della lunghezza

Analizziamo i calcoli del coseno:

$$\cos(\text{Sas, PaP}) \approx 0.789 \times 0.832 + 0.515 \times 0.555 + 0.335 \times 0.0 + 0.0 \times 0.0 \approx 0.94$$

$$\cos(\text{Sas, WH}) \approx 0.79$$

$$\cos(\text{Pap, WH}) \approx 0.69$$

Per calcolare il punteggio tramite coseno si utilizza il seguente algoritmo:

Algoritmo 2 — *CosineScore(q)*

1: *float* $Scores[N] = 0$
2: *float* $Length[N]$

```
3:  for each query term t
4:      do calculate w_{t,q} and fetch postings list for t
5:          for each pair(d, tf_{t,d}) in postings list
6:              do Scores[d] += w_{t,d} × w_{t,q}
7:  Read the array Length
8:  for each d
9:      do Scores[d] = Scores[d] / Length[d]
10: return Top K components of Scores[]
```

Il tf-idf weighting ha diverse varianti:

Term frequency		Document frequency		Normalization	
n (natural)	$\mathrm{tf}_{t,d}$	n (no)	1	n (none)	1
l (logarithm)	$1 + \log(\mathrm{tf}_{t,d})$	t (idf)	$\log \frac{N}{\mathrm{df}_t}$	c (cosine)	$\frac{1}{\sqrt{w_1^2+w_2^2+\ldots+w_M^2}}$
a (augmented)	$0.5 + \frac{0.5 \times \mathrm{tf}_{t,d}}{\max_t(\mathrm{tf}_{t,d})}$	p (prob idf)	$\max\{0, \log \frac{N-\mathrm{df}_t}{\mathrm{df}_t}\}$	u (pivoted unique)	$1/u$
b (boolean)	$\begin{cases} 1 & \text{if } \mathrm{tf}_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$			b (byte size)	$1/CharLength^{\alpha}$, $\alpha < 1$
L (log ave)	$\frac{1+\log(\mathrm{tf}_{t,d})}{1+\log(\mathrm{ave}_{t \in d}(\mathrm{tf}_{t,d}))}$				

Figura 14: Varianti del tf-idf weighting